Prototype

What the prototype is

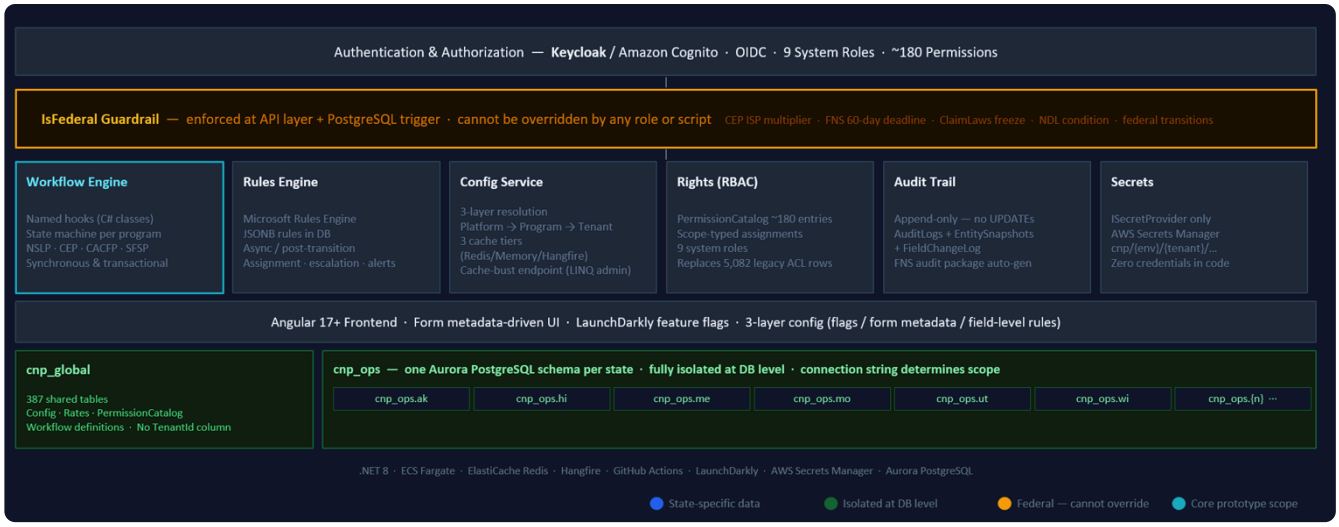
A working system running one state's application-to-approval workflow on the new platform, demonstrated live. Real API, real database, real workflow engine, real OIDC login. The demo state's actual program configuration must be in the system. A configuration change must be made live during the demo and propagate in under 10 minutes.

Hard constraints

- **16 weeks from kickoff to demo-ready** (August to early December). The nTIG window is external and fixed, so we'll be using other sources until prototype is ready.
- **One state only.** The prototype environment is configured for one demo state. No multi-state scenarios, no cross-state features.
- **No production data.** All data in the prototype environment is synthetic or anonymized. See Section 5.
- **Offshore: greenfield environment only, no existing LINQ system access.** This is a net-new build. Offshore engineers have no access to any existing LINQ system — not CTS, not DIS, not any current LINQ database, codebase, or infrastructure. They receive access only to the new GitHub repository and new AWS environments provisioned specifically for this project. The design documents in this folder are their specification.
- **Claims workflow is out of scope.** The prototype ends at application approval. No claims, no FNS report generation, no reimbursement calculations beyond the ClaimLaws freeze display.
- **Configuration admin UI is internal only.** One screen for the speed demo. Not a full admin application.

What must be demonstrably real

- OIDC login with real token issuance (not mocked)
- Workflow state machine evaluating actual named hook implementations
- Demo state's specific condition hook executing correctly
- ClaimLaws row created in the database at approval (not a display trick)
- Audit log entries written for every workflow transition (queryable in the DB)
- Config change propagating to the application layer within the cache TTL
- Role-based access enforced by the server, not just the UI





CNP_Architect... e1.pptx
15 Jun 2026, 11:50 AM

Tech Stack

Role	Production data	Staging data	Dev data	Production credentials	Schema migrations
LINQ US onshore engineer	Read-only via break-glass	Full access	Full access	None	Via CI/CD pipeline only
LINQ onshore config admin	Config writes via API	Full access	Full access	None	No
Offshore engineer (build team)	None — ever	Synthetic data only	Synthetic data only	None — ever	No
State agency administrator	Own state only, via app	None	None	None	No
FNS Auditor	Read-only, specific states, via app	None	None	None	No

LINQ DBA (US-based)	Read-only by default; break-glass write	Full access	Full access	Via secrets manager only	Via CI/CD pipeline only

"Break-glass" write: the engineer submits a request through the access management system, provides a business justification, access is granted for a time-limited session. This process applies to onshore engineers and DBAs. It is never available to offshore contractors.

Layer	Technology	Decision doc	Notes
Frontend	Angular 17+, LINQ snackpaq design system - Responsive. No IE support required? **Starting point: Shared Services Starter Angular project (includes snackpaq + standard company libraries).** https://github.com/K-12-Solutions-LINQ/angular-starterConnect Connect your Github account	—	Responsive. No IE support required.
API	.NET 8, MediatR (CQRS), FluentValidation	—	One API project. No microservices for prototype. Commands and Queries organized by domain subfolder, not flat. DTOs required at API boundary with no entity objects exposed to consumers.
Database	Aurora PostgreSQL (cnp_global +	ADR-005	cnp_ops schema per state: `cnp_ops.`

	cnp_ops)		`{state_code}`
Auth	Harmony & Amazon Cognito	—	Decision required Sprint 1. Keycloak preferred for demo flexibility; Cognito preferred for ops simplicity.
Cache	Amazon ElastiCache (Redis)	ADR-004 / CnpDefaults Design §8	Three-tier TTL per config key. Cache-bust endpoint required.
Secrets	AWS Secrets Manager via ISecretProvider	CNP_Secrets_Management.html	No credentials in code, config files, or environment variables. Hard rule.
Background jobs	Hangfire (PostgreSQL backing store)	CNP_Flow_Gaps.html	PY Rollover job registered but not required to fire for prototype.
Infrastructure	AWS ECS Fargate, ALB, VPC	—	Single region (us-east-1) for prototype. No multi-region until DR plan is finalized.
CI/CD	GitHub Actions	—	Deploy to staging on merge to main. Deploy to demo env on tag.
Migrations	FluentMigrator or EF Core Migrations	ADR-005	Per-schema migrations. cnp_global runs once; cnp_ops.{state} runs per state.
DB triggers	PostgreSQL PL/pgSQL	ADR-007, Workflow Engine §9	IsFederal enforcement triggers on TenantTransitionOverri

			des, TenantConfigOverrides.
Feature flags	LaunchDarkly	CNP_ERD.html	Module-level flags only for prototype. ~5 flags needed.

Algoworks Team

Name	Role	email
Sourav Ghosh	Project Manager	sourav.ghosh@algoworks.com
Tosika Mukherjee	Solution / Technical Architect	tosika.mukherjee@algoworks.com
Atul Matvar	Data Engineer	atul.matvar@algoworks.com
Suraj Singh	DevSecOps & Platform Lead	suraj.singh@algoworks.com
Robin Kumar	Backend Engineer 1	robin.kumar@algoworks.com
Vinoth K	Backend Engineer 2	vinod.k@algoworks.com
Ashish Nigam	Backend Engineer 3	ashish.nigam@algoworks.com
Ashutosh Singh	Frontend Engineer (UI/UX) 1	ashutosh.singh@algoworks.com
Umang Gupta	Frontend Engineer (UI/UX) 2	umang.gupta@algoworks.com
Pratibha Tanwar	QA Engineer 1	pratibha.tanwar@algoworks.com
Abhilash Soni	QA Engineer 2	abhilash.soni@algoworks.com
Reshmi Parvin	Junior AI 1	Reshmi.parvin@algoworks.com
Sairajendra Jagdale	Junior AI 2	sairajendra.jagdale@algoworks.com

PR reviews

These are enforced in the GitHub branch protection rules.They are not optional:

- Any file in `/src/Hooks/`, `/src/Compliance/`, `/src/Audit/`, or `/migrations/` → onshore tech lead required as approver
- Any PR modifying `ConfigDefinitions` seed data or `WorkflowTransitionConditions` seed data → onshore tech lead required
- Any PR that adds a new `IsFederal=true` row must include a corresponding guardrail assertion test in the same PR. This is enforced by a GitHub Actions status check that scans the diff and fails if no test file is added or modified. The check is required for merge. The onshore tech lead may apply the `guardrail-test-exempt` label to bypass the check when a test is genuinely not applicable and the label application is logged. Offshore engineers cannot apply this label.
- Offshore engineers cannot self-approve any PR

These rules are enforced via .github/CODEOWNERS. GitHub automatically adds the onshore tech lead as a required reviewer on any PR touching `/src/Hooks/`, `/src/Compliance/`, `/src/Audit/`, or `/migrations/`. The CODEOWNERS file is owned by the onshore engineers (managers?) and is itself a protected path.

Environments

All environments below are **new AWS accounts/resources provisioned specifically for this project**. None of them are connected to any existing LINQ system, database, or infrastructure. Offshore engineers have no path to any existing LINQ system.

Environment	Purpose	Data	Offshore Access	Deploys From
Dev	Individual developer environment. New greenfield AWS account. Ephemeral, spun up per developer.	Synthetic only. Seed script generates fake applications, sponsors, sites. Generated from scratch — no CTS/DIS data.	Full — this is a new environment with no LINQ data. Offshore engineers work here.	Local / feature branch
Staging	Integration testing. All PRs to main deploy	Synthetic only. Nightly data reset from seed	Read for troubleshooting. No	Merge to main (GitHub

	here automatically. New greenfield environment.	script.	direct DB write access.	Actions)
Demo	The prototype environment. Stable. No uncontrolled deploys.	Synthetic data curated to match demo state profile. No real state data. No CTS/DIS data.	None	Tagged release only. Onshore tech lead approves every deploy.
UAT	Demo state informal UAT (Weeks 13–15). Isolated from demo env.	Synthetic data curated to match demo state's real program profile (not real records).	None	Tagged release (same tag as demo env)

Infrastructure checklist

- [ ] AAWS account provisioned with ECS task roles; no static keys
- [ ] VPC (us-east-1) with private app subnets and protected data subnets; NAT Gateway for outbound
- [ ] Aurora PostgreSQL — single shared cluster (dev/staging)
- [ ] ElastiCache Redis — single node sufficient for prototype
- [ ] ECS Fargate cluster with task definition for the CNP API (ASP.NET Core, an open-source web development framework | .NET Core 10)
- [ ] ECR repository for container images
- [ ] ALB routing to Fargate with health checks
- [ ] ACM certificate for demo domain; CloudFront + S3 for Angular SPA
- [ ] Cognito for user auth and M2M; Harmony JWT Middleware configured for JWKS stateless validation
- [ ] Secrets Manager paths: `cnp/staging/...` and `cnp/demo/...`; ECS injection only
- [ ] LaunchDarkly project with dev and demo environments
- [ ] GitHub Actions workflows: build → test → deploy to staging on PR merge; deploy to demo on `demo-*` tag; onshore approval gate
- [ ] CloudWatch for basic logs and alerts
- [ ] Synthetic data generation script runnable with one command
- [ ] Offshore GitHub accounts added to Modernization-Algoworks team; repo access only, no shared accounts

Data Strategy

Synthetic data generation

All environments use synthetic data. The generation script must produce a consistent, realistic demo dataset:

- **Sponsors:** 5–10 fake sponsors with realistic names and addresses in the demo state

- **Sites:** 20–40 schools or sites per sponsor, with realistic enrollment numbers
- **Applications:** Mix of statuses — Draft, Submitted, UnderReview, Approved, Declined. Enough to make the list view look real.
- **Users:** 3 Caseworker accounts, 2 Reviewer accounts, 1 StateAgencyAdmin account, 1 LINQConfigAdmin account (for speed demo)
- **Config values:** Demo state's CnpDefaults seeded from the three-layer hierarchy. All ~180 ConfigDefinitions keys seeded at platform level; demo state overrides for known customizations.
- **Rates:** Current SY reimbursement rates in PovertyGuidelineHistory. These are public FNS data — use real current rates.

Demo state workflow seed

The demo state's workflow must be seeded from the actual design documents, not improvised:

- WorkflowStates for the application lifecycle (Draft → Submitted → UnderReview → Approved → Declined)
- WorkflowTransitions with all required conditions and actions
- At minimum: `FreezeClaimLaws` action on ApproveApplication transition (IsFederal=true)
- At minimum: `AllClaimErrorsResolved` condition on SubmitApplication
- Demo state-specific hook: TBD based on demo state selection — one real customization that differs from the default workflow. Must be identified before Sprint 3.
- IsFederal triggers deployed and tested before any seed data loads

Speed demo configuration change

The specific config key and value change for the speed demo must be identified before Sprint 6. Requirements:

- The change must be visible in the application workflow (changes something the state contact will recognize)
- It must be in a Tier 1 or Tier 2 cache key (propagates within 5 minutes, not midnight)
- It must NOT be an IsFederal=true key (the demo shows flexibility, not a block)
- Example candidates: application submission deadline threshold, notification email subject line, a validation message text, a display label

14 weeks of active build + 2 weeks of freeze and demo prep. Sprint velocity assumes 2 offshore backend engineers + 1 offshore frontend engineer + 1 offshore QA, with onshore oversight.

Offshore engineers work exclusively in the new greenfield dev/staging environments — no access to any existing LINQ system at any point.

Sprints

Sprint 1: Infrastructure & Environments Weeks 1–2 · August 17-28

Deliverables

- Onshore tech lead copies http://github.com/K-12-Solutions-LINQ/angular-starter into the new greenfield repo as the initial commit, before offshore access is provisioned. Offshore gets it through the approved channel.
- AWS environments (dev, staging) provisioned with VPC, subnets, security groups
- Aurora PostgreSQL cluster (dev/staging) live with connection confirmed
- ElastiCache Redis cluster live
- ECS Fargate cluster with placeholder task definitions
- Secrets Manager paths established and documented
- GitHub Actions: build + unit test on PR; deploy to staging on merge to main; IsFederal guardrail check (blocks merge if `IsFederal=true` added without a test). `.github/CODEOWNERS` configured and onshore tech lead auto-added as required reviewer on `/src/Hooks/`, `/src/Compliance/`, `/src/Audit/`, `/migrations/`.
  - isfederal-guardrail.yml Drop this file into `.github/workflows/` in the new repo. Then in GitHub branch protection for `main`, add `IsFederal=true → Guardrail Test Required` as a required status check.

    What it does:

    - Scans the PR diff for any added line containing `IsFederal=true` (catches C#, SQL, and JSON seed formats)
    - If found and no test file is in the PR → **blocks merge** with a clear error message telling the author exactly what to do
    - If the `guardrail-test-exempt` label is applied → passes, but writes a timestamped audit entry to the Actions run summary so there's a permanent record of every bypass
    - Offshore engineers can't apply that label (requires Triage+ repo role)
- .github/CODEOWNERS file created and branch protection rules configured to require CODEOWNERS review.
  - Verify test PR touching `/migrations/` automatically adds onshore tech lead.
- Keycloak deployed to staging ECS, admin console accessible onshore only
- Synthetic data generation script runnable (`npm run seed` or equivalent)

- Offshore individual accounts provisioned for dev environment only
- LaunchDarkly project + staging environment created

Owners

- DevOps: all AWS provisioning
- Onshore Tech Lead: Keycloak setup, secrets paths, offshore access provisioning
- Offshore Backend Lead: synthetic data script, repository scaffold

Gate: Before offshore team receives anything: (1) GitHub repo exists and is isolated from all existing LINQ repositories, (2) dev environment AWS account has no VPC peering or connectivity to any existing LINQ account, (3) offshore individuals have accounts — not shared credentials, (4) design document package exported and delivered to offshore team. Onshore tech lead and DevOps both sign off. This is the offshore team's kickoff gate — they do not begin until these are confirmed.

Sprint 2: Database Schema & Platform Services Weeks 3–4 · August 31–September 1

Deliverables — Database (Layer 1)

- cnp_global schema migrations: LookupCodes, FederalProgramDefinitions, PovertyGuidelineHistory, ConfigDefinitions, WorkflowEntityTypes, WorkflowStates, WorkflowTransitions, WorkflowTransitionConditions, WorkflowTransitionActions, PermissionCatalog, RoleDefinitions, RolePermissions
- cnp_ops.{demo_state} schema created and confirmed isolated
- IsFederal trigger `trg_block_federal_config_override` deployed and tested
- IsFederal trigger `trg_block_federal_transition_override` deployed and tested
- Database roles: `cnp_app`, `cnp_migration`, `cnp_readonly` created
- ConfigDefinitions seeded: ~20 key entries covering application workflow (full ~3,000 key seed in later sprint)
- PovertyGuidelineHistory seeded: current SY rates (public FNS data)

Deliverables — Platform Services (Layer 2)

- .NET 8 API project scaffold: MediatR pipeline, FluentValidation behaviors, exception middleware
  - MediatR folder structure established: `/src/Commands/{Domain}/` and `/src/Queries/{Domain}/`, not a flat Commands/ dump. Domains for prototype: Applications, Workflow, Config, Rights.
  - DTO pattern enforced: every query response and command result returns a purpose-built DTO. No EF Core entity or DB model objects cross the API boundary. Reviewed by offshore

backend lead before Sprint 3.

- `ICnpConfigService`: three-layer resolution (no Redis yet — in-memory or direct DB), returning typed values
- `ICnpPermissionService`: `HasPermission(userId, tenantCode, permissionCode)` resolving from DB
- JWT validation middleware: extracts roles + tenant scopes from Keycloak token
- Tenant context middleware: sets `cnp_app` search_path to `cnp_ops.{state_code}` per request
- Healthcheck endpoint: `GET /health` confirming DB + Redis connectivity
- Keycloak realm configured: demo state, test users (Caseworker × 3, Reviewer × 2, StateAgencyAdmin × 1, LINQConfigAdmin × 1)

Gate: Onshore tech lead verifies (1) IsFederal triggers reject a test write to a federal-locked key, (2) tenant context middleware correctly sets schema per JWT claim, (3) offshore team cannot connect to staging from their dev environment. All three must pass.

Sprint 3: Workflow Engine & Named Hooks Weeks 5–6 · September 14–25

Deliverables

- `WorkflowEngine` service: resolves current state, evaluates transition conditions (named hooks), executes transition actions, persists new state, publishes domain event
- Named hook registry: `IWorkflowCondition` and `IWorkflowAction` interfaces, DI registration, catalog endpoint `GET /workflow/hooks`
- Hook implementations (onshore tech lead must review all):
  - `FreezeClaimLaws` — action, IsFederal=true: creates ClaimLaws snapshot row at approval
  - `AllClaimErrorsResolved` — condition: checks no open ApplicationErrors on record
  - `NoNDLMatch` — condition, IsFederal=true: stub returning true (full NDL integration is post-prototype)
  - *Demo state hook* — TBD by onshore config specialist before this sprint begins
- Demo state workflow seeded: WorkflowStates, WorkflowTransitions, conditions, actions for the application lifecycle
- TenantTransitionOverrides seeded: demo state customization applied via override (not by modifying base workflow)
- Unit tests: each hook has a test class. 100% coverage of condition/action logic.

- IsFederal guardrail assertion tests: `NoNDLMatch` cannot be removed via API or direct DB write

Pre-sprint requirements

- Demo state hook identified by onshore config specialist before Sprint 3 starts. If not identified, Sprint 3 proceeds without it and it is added in Sprint 4 — acceptable.
- Onshore tech lead has reviewed the WorkflowEngine design doc (CNP_Workflow_Engine.html) with the team. All engineers understand named hooks vs. expression language (ADR-003).

Onshore review required

- All named hook implementations
- All seed data for WorkflowTransitionConditions with IsFederal=true
- IsFederal guardrail assertion test results

Gate: Guardrail assertion tests pass: (1) FreezeClaimLaws cannot be removed from ApproveApplication transition via API, (2) direct DB attempt to delete a federal-IsFederal=true condition raises PostgreSQL exception, (3) exception is logged to AuditLogs. Onshore tech lead signs off before Sprint 4 begins.

Sprint 4: Application Domain Weeks 7–8 · September 28–October 9

Deliverables

- Applications table + ApplicationSponsors + ApplicationSites migrations (cnp_ops schema)
- AuditLogs, EntitySnapshots, FieldChangeLog tables (cnp_ops schema)
- ClaimLaws table (cnp_ops schema)
- Application commands: SubmitApplication, TransitionApplication (fires WorkflowEngine), AddApplicationError, ResolveApplicationError
- Application queries: GetApplicationList (paged, filtered by status), GetApplicationDetail, GetApplicationAuditTrail
- Every command writes to AuditLogs + EntitySnapshots. Every field change writes to FieldChangeLog.
- ClaimLaws row created by FreezeClaimLaws action hook — verified by integration test
- Seed data: 30–50 synthetic applications in various states in the demo state schema
- Integration tests: full application lifecycle (Submit → Review → Approve) passing with WorkflowEngine
- Guardrail assertion test: ClaimLaws rows have no UPDATE endpoint — attempt returns HTTP 405

API contracts (to be finalized by end of sprint)

- `GET /applications?status=&page=&pageSize=`
- `POST /applications`
- `GET /applications/{id}`
- `POST /applications/{id}/transitions` — body: `{ "toState": "Submitted" }`
- `GET /applications/{id}/audit`
- `GET /applications/{id}/claim-laws`

Frontend Sprint 5 builds against these contracts. They must not change after Sprint 4 ends.

Gate: Integration test passes the full lifecycle: Submit → transition to UnderReview → transition to Approved. Confirm ClaimLaws row exists in DB post-approval. Confirm AuditLogs + EntitySnapshots rows exist for every transition. API contracts published and frozen for frontend team.

Sprint 5: Frontend — Shell, Auth, Application List Weeks 9–10 · October 12–23

Deliverables

- Angular 17+ project with LINQ snackpaq design system integrated
- OIDC login flow: redirects to Keycloak, receives token, stores in memory (not localStorage — not supported in demo env)
- Route guards: unauthenticated users redirected to login; unauthorized roles redirected to access-denied page
- Navigation: role-based. Caseworker sees: Applications. Reviewer sees: Applications + Review Queue. LINQConfigAdmin sees: Config Admin (hidden from other roles).
- Application list screen: paged table, status filter, search by sponsor name. Loads real data from API.
- Application list: Caseworker sees own-state applications; Reviewer sees full queue for state. Role enforced by API, not just UI.
- Loading states, error states, empty states — all handled. No console errors visible in demo.
- Responsive layout (desktop priority for demo; tablet acceptable).

Design decisions

- No localStorage or sessionStorage — token stored in service memory, re-login on refresh is acceptable for prototype
- HTTP interceptor attaches Bearer token to all API requests
- HTTP interceptor handles 401 (redirect to login) and 403 (show access-denied message)
- Angular environment files contain API base URL only — no credentials, no secrets

Onshore config specialist input needed

- Demo state program names and labels for the application list columns
- Demo state status names (may differ from default labels)

Gate: Full login → application list flow works end-to-end in staging. Log in as Caseworker: see applications. Log in as Reviewer: see review queue. Log in as LINQConfigAdmin: see Config Admin link. Verify 403 if Caseworker attempts to access Reviewer-only endpoint directly (not just hidden in UI).

Sprint 6: Frontend — Workflow, Audit Trail, Speed Demo Screen Weeks 11–12 · October 26–November 6

Deliverables

- Application detail page: all fields, status badge, available action buttons (driven by current workflow state + user role)
- Workflow action buttons: Submit, Approve, Decline, Hold — only available actions shown based on current state and user role. Unavailable actions hidden, not disabled.
- Approval screen: after approval, displays ClaimLaws summary ("Rates locked at approval: Free $X.XX, Reduced $X.XX, Paid $X.XX for NSLP") fetched from `GET /applications/{id}/claim-laws`
- Audit trail panel: collapsible. Shows every transition with actor, timestamp, from-state, to-state. Formatted for a human, not a developer.
- Demo state-specific field: whatever the demo state's customization is, it must be visible in the application detail UI
- Redis cache integrated in `ICnpConfigService`: Tier 1 keys (5-min TTL), Tier 2 (request-scoped), Tier 3 (nightly). Cache-bust endpoint live.
- Config Admin screen (LINQConfigAdmin role only): single config key editor. Shows current value, allows edit, calls cache-bust on save, confirms propagation. Used for speed demo — must be polished.

Speed demo prep (onshore config specialist)

- Identify the specific config key to demonstrate (Tier 1 or Tier 2)
- Confirm change is visible in the application workflow UI
- Time the full cycle: make change in Config Admin → navigate to application → confirm change visible. Must be under 5 minutes.
- Document the "legacy equivalent" story: how long would this change take on the current system?

Redis integration checklist

- `CacheTier` column populated on ConfigDefinitions seed data
- Cache-bust endpoint: `POST /admin/config/cache/bust` requires LINQConfigAdmin role, logged to AuditLogs
- Config write service calls cache-bust automatically when a TenantConfigOverride is saved

Gate: Speed demo dry run: onshore tech lead + config specialist run the full demo flow start to finish. Change is made in Config Admin. Change is visible in application workflow within 5 minutes. Audit log shows the cache bust event. Time the full sequence. If >8 minutes, investigate before Sprint 7.

Sprint 7: Demo State UAT & Data Validation Weeks 13–14 · November 9–20

Deliverables

- UAT environment provisioned from Sprint 6 tagged release
- Curated synthetic dataset: 50–100 applications matching the demo state's real program profile (correct program types, realistic enrollment sizes, realistic status distribution)
- Demo state informal UAT: 2–3 people from demo state agency access UAT env and walk through their primary workflows. LINQ account team facilitates. 2-hour session, feedback captured.
- Issue log from UAT: all feedback triaged (fix before demo / fix post-demo / won't fix)
- High-priority UAT issues fixed in this sprint
- Small-scale workflow validation: run 10–20 synthetic applications through the full lifecycle, confirm outcomes match the expected results documented in the workflow design specs. Full replay against real CTS/DIS data is onshore-only work done during state migration — not part of the greenfield prototype.
- Internal dry-run demo: onshore tech lead + account team + engineering. Full demo script (from CNP_October_Plan.html). Timed at 30 minutes or less.
- IsFederal guardrail regression suite: full suite passing on Sprint 7 release tag

What to watch for in UAT

- Any workflow step where the state contact says "that's not how we do it" — investigate: is it a missing customization, a config gap, or a design difference?
- Performance: application list loads in <2 seconds. Application detail loads in <1 second. Anything slower gets a ticket.
- Any visible errors or console errors during the walkthrough — zero tolerance in the demo environment
- Labels and terminology: does everything use the demo state's preferred language?

Not expected from this sprint

- No new features — this is a fix sprint only
- No claims workflow work
- No FNS report work

Gate: Demo state contact confirms UAT is satisfactory. Internal dry-run demo completes in ≤ 30 minutes with no technical failures. All high-priority UAT issues closed. IsFederal guardrail suite passing. Onshore tech lead approves tagging the demo environment release.

Sprint 8: Demo Environment Freeze & Final Prep Weeks 15–16 · November 23–December 4

Deliverables

- Demo environment deployed from Sprint 7 approved tag. Freeze enforced: no code changes after deploy.
- Demo environment smoke test: full demo script run by someone who has not seen it before. Confirm it works without coaching.
- Performance validation: application list <2s, application detail <1s, approval transition <3s, config change propagation <5min. All measured from demo environment, not staging.
- Demo data final check: all synthetic applications look realistic, no placeholder names like "Test Sponsor 1," no obviously fake data visible in the demo flow
- Backup plan documented: if demo environment has a failure during a live demo, what is the fallback? (Staging environment on identical tag as backup.)
- Account team briefed on demo script, Q&A answers, and what questions to escalate vs. answer themselves
- Demo invite list confirmed: state contacts, dates, locations or video call links

Remaining mid-priority UAT issues

- Mid-priority issues from Sprint 7 UAT: fixed in staging, deployed to demo env before freeze, or deferred to post-prototype
- No new features in Sprint 8 regardless of what is requested

What offshore team works on in Sprint 8

- Post-Prototype backlog: begin Sprint 1 work for the next phase (FNS reports, claims workflow, second state configuration)
- No offshore changes to demo environment — freeze is absolute

Gate: Demo environment passes smoke test by someone unfamiliar with it. Performance benchmarks met. Onshore tech lead formally declares "demo-ready." First state demo

scheduled.

Definition of Done

Per story / ticket

- Code compiles without warnings
- Unit tests written and passing
- Integration test covers the happy path and at least one failure path
- No new console errors in staging
- API contract matches the frozen contract for that sprint (no unannounced breaking changes)
- If compliance-related: onshore tech lead has reviewed and approved the PR
- Deployed to staging and smoke tested

For the prototype to be demo-ready

Requirement	Verification
Login works with demo Caseworker, Reviewer, and LINQConfigAdmin credentials	Manual smoke test
Application list loads with realistic synthetic data in <2 seconds	Stopwatch on demo environment
Full application workflow: Submit → UnderReview → Approve completes without error	End-to-end integration test + manual
Demo state-specific hook fires correctly during workflow	Automated integration test + visible in audit trail
ClaimLaws displayed on approval screen with correct rates	Manual verification against PovertyGuidelineHistory seed
Audit trail shows all transitions with actor, timestamp, state change	Manual verification in UI + DB query
Config change in Config Admin screen propagates to application workflow in <5 minutes	Timed during dry-run
Role switch: Caseworker vs Reviewer see different navigation and different data access	Manual smoke test both roles

Server-side role enforcement: Caseworker cannot access Reviewer-only endpoint via direct API call	Automated test: `GET /applications/review-queue` with Caseworker token → HTTP 403
IsFederal guardrail assertion suite: all 7 required assertions passing	Automated, CI/CD
No real state data in demo environment	Data review by onshore config specialist
Demo environment stable for 48 hours with no crashes or restarts	ECS task restart count = 0 over 48 hours

Design Document Index

Every engineering decision for the prototype has a design document. Engineers must read the relevant doc before implementing the corresponding component. If there is a conflict between this build plan and a design document, the design document wins — flag the conflict, don't resolve it silently.

Component	Document	Key Sections
Workflow state machine, named hooks, IsFederal triggers	Workflow Engine	Full table design, application lifecycle states, override model, §9 DB-level trigger pseudocode, database roles
Rules Engine (escalation, assignment routing — post-transition)	Rules Engine	Boundary with Workflow Engine (see ADR-010). Named hooks = synchronous/transactional; Rules Engine = async/post-transition.
Config service (three-layer resolution, cache tiers, cache-bust)	Configuration Defaults	ICnpConfigService, three-layer hierarchy, cache tiers (Immediate/Session/Nightly), cache-bust endpoint

Rights model (RBAC, 9 roles, PermissionCatalog)	CNP Rights	9 system roles, PermissionCatalog, HasPermission, ScopeType, TenantRoleOverrides
Secrets management (ISecretProvider, naming convention, hard rules)	Secrets Management	8 hard rules, ISecretProvider interface, `cnp/{env}/{tenant}/{category}/{name}` naming
Architecture decisions (IsFederal, RBAC, append-only audit, named hooks, database-per-tenant, DIS+CTS unification, NLP template catalog, Rules Engine boundary)	Architecture Proposals	ADR-001 through ADR-010. Read the relevant ADR before implementing any component it covers.
ERD (all table definitions, domain groupings, FK chains)	Data	Reference for all table names, columns, relationships. ProgramGroupYears spine, 25 domains across 4 tiers.
Federal math (ClaimLaws freeze, CEP, CACFP, SFSP, rate calculation)	The Math	ClaimLaws freeze pattern, CEP ISP × 1.6 multiplier, CACFP tier determination, SFSP single rate
Annual rate updates (ClaimLaws versioning, FNS rate change process)	Annual Rate Updates	Note: FNS renamed to FNA effective June 1, 2026. Report numbers unchanged. ClaimLaws freeze pattern.
Business flow gaps (NDL check, PY Rollover, FNS-640/905)	CNP Business Flow Gaps	NDL check design, PY Rollover tables (informational for prototype — not in scope until production go-live)

Offshore/onshore boundary (5 Configuration APIs, zero-deployment onboarding)	Code Deployment vs Configuration	What engineering builds vs. what configuration operates. The speed demo lives in the Configuration layer.
Frontend architecture (Angular, feature flags, form metadata, field rules)	Frontend Architecture	Three layers of configuration (LaunchDarkly flags, form metadata, field-level rules). Do not conflate them.
Domain model overview (25 domains, 4 tiers, service decomposition)	Platform Domains	High-level domain map. Orient here before diving into component-level docs.
Offshore access controls, data handling, NIST 800-171	Access Governance & Data Handling Policy	§4 offshore policy, §3 access matrix. Must be acknowledged by all offshore contractors before access is provisioned.
Infrastructure	Infrastructure Requirements	Secrets Manager naming convention, the two DB roles, the environment provisioning checklist, and — importantly — it names the actual preferred auth provider (LINQ Harmony Auth)
Testing requirements (guardrail assertions, federal math tests, per-PR gates)		§4 IsFederal guardrail assertions (7 required), §8 per-PR gates. QA engineer reads this first.
Data Flow diagram for the system.	Data Flow Document	Auth lane browser login → WAF/API GW → Core API redirect → Harmony Auth login page → JWT (Bearer) issuance → subsequent API requests validated against that JWT with CNP RBAC applied.